



INF573 FINAL PROJECT

Distance and speed estimation on a car dashcam video

December, 2023

Pierre Emmanuel BAVIERE - Matthieu DESTRADE



CONTENTS

1	Introduction	3
2	Inspirations and reference works	4
3	Implemented method	4
3.1	Detection of vehicles (YOLO)	4
3.2	Distance calculation	6
3.3	Vanishing point detection (OpenCV,RANSAC)	7
3.4	License plate detection (YOLO, OpenCV)	8
3.5	Pooling and calibration	9
4	Results	10
4.1	Images	10
4.1.1	Detection of vehicules	10
4.1.2	Vanishing point detection	11
4.1.3	License plates detection	12
4.2	Videos	13
5	Conclusion	15

1

INTRODUCTION

Autonomous cars are a widely researched topic due to their numerous potential applications in terms of personal comfort as well as optimization of transportation time, reduction of energy costs, and limitation of accidents. Currently, most autonomous vehicles require the use of complex integrated sensors, such as radar or LiDAR, to calculate distances from surrounding objects.

It would therefore be interesting to be able to calculate the distances and speeds of surrounding objects solely from a **traditional camera video**. For instance, this would allow an everyday car user to place a camera on their dashboard, record the road ahead, and retrieve data that could potentially be shared with other systems in the car.

The desire to allow great freedom in installing such a system imposes several constraints on the algorithm responsible for calculating distances and speeds in the video. This means that while the camera's characteristics can be assumed to be **fully known**, the same cannot be said for its **final position**. The algorithm must be capable of autonomously determining the camera's position and calibrating itself to provide an accurate estimation of distances.

In this report, we propose to describe our method of implementing such an algorithm. Our approach is based on three main phases: **detecting displayed vehicles in a video**, **identifying the vanishing point** of the road, and finally, calibrating distances through **license plate detection**.

2

INSPIRATIONS AND REFERENCE WORKS

We have based our work on several references. To begin with, we have been impressed by the performance of **YOLO** for real-time car detection, as illustrated by [5]. This demonstration convinced us to use deep learning for the detection and classification of objects in video frames.

Various studies have also inspired us to detect the vanishing point of the road on the horizon. The concept of utilizing the vanishing point of the road was introduced to us by [1]. The work of [2] then provided guidance on the overall structure of the algorithm needed to do so, and insights from [3] directed us towards the use of the **RANSAC method**, for which we found examples on GitHub.

As for the license plate detection method, it was partly inspired by the findings presented in the article by [4].

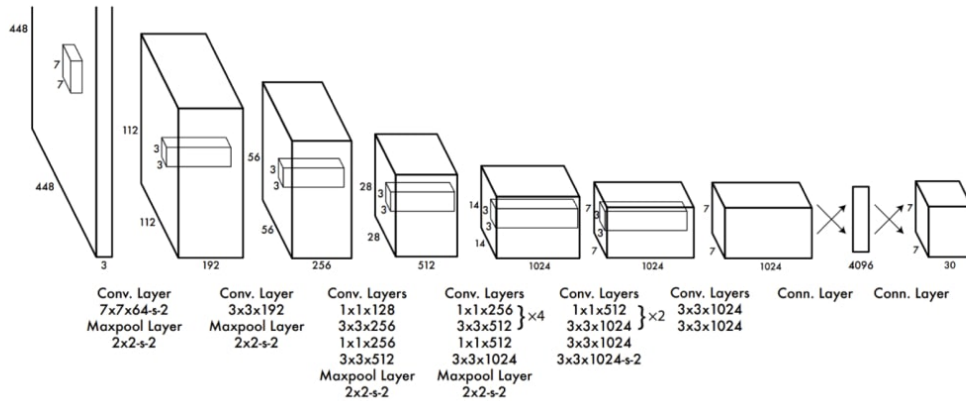
3

IMPLEMENTED METHOD

3.1 DETECTION OF VEHICLES (YOLO)

Image analysis involves many different tasks. We distinguish **detection** (framing a specific object or element in an image), **classification**, **image segmentation**, and **object tracking**. This task, which is simple for the human brain, is actually very complex for a computer. An approach using a neural network seems suitable for the problem. In 2014, Ross Girshick and the Microsoft team developed the **R-CNN model** using convolutional layers to process image information, perform detection, and then classification. Later, improvements were made, particularly in training the network to make it more efficient. These enhancements resulted in the fast, faster, and mask R-CNN models. However, the R-CNN model has major drawbacks ([6]). It is **slow** to perform analysis (about 45 seconds per image). The computation time is notably linked to the double successive use of the image (two-shot object detection). First, the model makes a coarse detection, and then, in a second step, the prediction is refined, and the detections are classified.

The **YOLO model** is a deep neural network with a very different operation. Instead of extracting regions of interest for analysis, the YOLO model performs classification and prediction in a single step, using a fully connected layer [10]. This behavior is the origin of the name YOLO: **you look only once**. One drawback of this model is that it practices spatial divisions of the image that limit the detection of small objects ([7]).



The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

Figure 1: YOLO v1 neural network architecture

Subsequently, the YOLO model has been improved up to version 8, currently in use. This model is typically trained on the **COCO dataset**.



Figure 2: COCO dataset image segmentation example

The model is extremely efficient, as the analysis is fast enough to be used on real-time videos. Additionally, the COCO dataset includes 79 different classes such as 'car', 'motorcycle', 'bus', and 'truck'. Finally, YOLO v8 is also capable of tracking its detections in a sequence of images. For these reasons, we have **chosen to use this model** for vehicle detection.

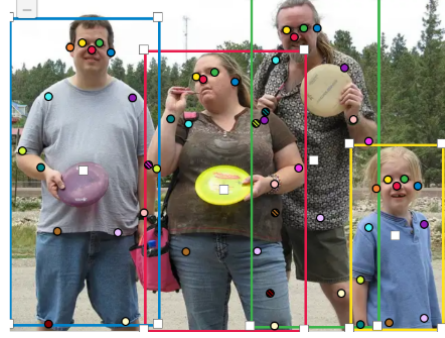


Figure 3: COCO dataset object detection and classification

3.2 DISTANCE CALCULATION

Once the vehicles were detected, the next step was to assess their distance from the camera. To do this, drawing inspiration from previous research, we exploited the fact that the detected objects were positioned on a flat road, whose precise orientation could be determined by finding its **vanishing point**.

Here, we assume that the camera used performs a perfect **perspective projection** (if not, one could consider correcting the image in preprocessing using calibration methods). Thus, we have:

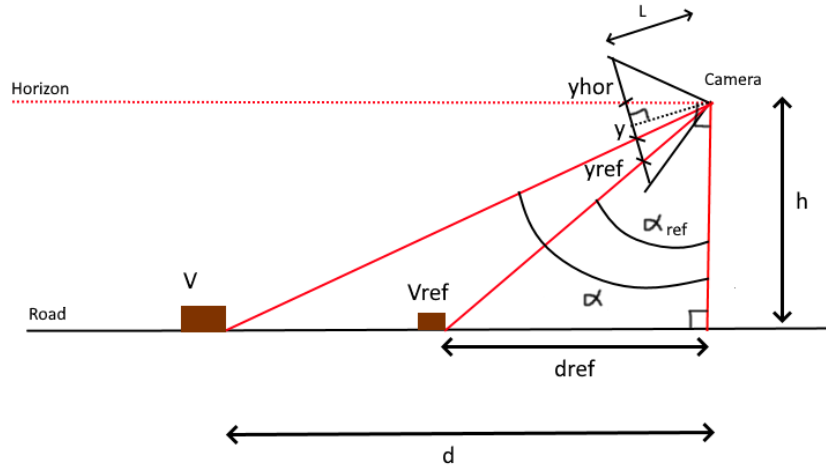


Figure 4: diagram of our situation

According to the diagram, we have (using degrees, taking the origin of coordinates for the y -axis as the center of the image, and orienting it towards the bottom):

$$90 - \alpha = \arctan \frac{y}{L} - \arctan \frac{y_{hor}}{L} \quad \text{and} \quad 90 - \alpha_{ref} = \arctan \frac{y_{ref}}{L} - \arctan \frac{y_{hor}}{L}$$

(L is the distance between the center of the camera and the projection plane).

So

$$\tan(90 - \alpha) = \frac{\frac{y}{L} - \frac{y_{hor}}{L}}{1 + \frac{y}{L} \frac{y_{hor}}{L}} = \frac{y - y_{hor}}{L + \frac{yy_{hor}}{L}} \quad \text{and} \quad \tan(90 - \alpha_{ref}) = \frac{y_{ref} - y_{hor}}{L + \frac{y_{ref}y_{hor}}{L}}$$

Moreover :

$$\tan \alpha = \frac{d}{h} \quad \text{and} \quad \tan \alpha_{ref} = \frac{d_{ref}}{h}$$

Therefore :

$$\begin{aligned} d &= \tan \alpha h \\ &= \frac{\tan \alpha}{\tan \alpha_{ref}} d_{ref} \\ &= \frac{\tan 90 - \alpha_{ref}}{\tan 90 - \alpha} d_{ref} \\ &= \frac{y_{ref} - y_{hor}}{y - y_{hor}} \frac{1 + \frac{yy_{hor}}{L^2}}{1 + \frac{y_{ref}y_{hor}}{L^2}} d_{ref} \end{aligned}$$

In practice, we will assume that the camera view axis is almost **aligned with the road** and that $\frac{y_{hor}}{L} \approx 0$. We then have:

$$d \approx d_{ref} \frac{y_{ref} - y_{hor}}{y - y_{hor}}$$

One could get rid of this approximation by having a camera with **known data** corresponding to L . To find this, one could, for example, experimentally determine the camera's angular aperture and then apply a trigonometric formula similar to the ones used earlier in order to get L .

In any case, we observe that it is possible to determine the distances of vehicles **along the view axis** under two conditions:

- Knowing the **position of the vanishing point of the road** in the image.
- Knowing the **position of a vehicle** on the road. (V_{ref}).

In the following steps, we therefore detect these two elements.

3.3 VANISHING POINT DETECTION (OPENCV,RANSAC)

To find the vanishing point of the road, we used the fact that most lines in the image **converge towards it**. Indeed, the lines in the image primarily consist of those painted on the road and of the lines of buildings around the road. In general, these lines are parallel to the road and converge towards the same vanishing point.

To detect the lines, following preprocessing on the image, including the application of a **Canny filter** for edge detection, we applied the OpenCV `HoughLinesP` algorithm. For each potential line in the image, this method counts the number of contour points that belong to it. If this count exceeds a threshold, it is likely that the line exists and is formed by the associated points.

Once the lines are extracted from the image, the next step is to determine their **point of intersection** to find the horizon. In reality, this is not straightforward because there are unwanted lines that do not converge towards this point, such as vertical lines from traffic signs or even lines of random directions from tree branches. To determine the most probable vanishing point, the one towards which most lines converge, we use the **RANSAC method**.

RANSAC stands for **RANdom SAMple Consensus**. It is an algorithm for estimating the parameters of a model while eliminating outliers. In our case, it works as follows:

- **Selection of a random pair of lines** and calculation of their **point of intersection**.
- Evaluation of the **distance** between the found point and each of the other lines in the image. If this distance is sufficiently small, the line is added as an **inlier**.
- If the number of inliers is sufficiently large, or at least larger than that of the points tested previously, we consider the found point of intersection as **the vanishing point**.

This procedure is repeated a finite number of times. The larger this number, the more likely it is to find a suitable vanishing point.

To avoid too much variability in the results, in practice, we processed the lines found in the image before applying the RANSAC method. We **removed vertical and horizontal lines** (signs, buildings, other vehicles...) and lines that were heading **towards points far from the center of the image** (where the vanishing point is presumed to be if the camera is approximately parallel to the road, as assumed in the previous calculations).

With the vanishing point thus detected, the next step is to determine the position of a vehicle on the road.

3.4 LICENSE PLATE DETECTION (YOLO, OPENCV)

To calculate the position of an object on the road based on its vertical coordinate in the image, we need to find a reference point in the image (to set d_{ref}). For this, we had the idea of using a standardized object that is likely to be filmed by the camera under driving conditions: **a license plate**.

Therefore, we sought to detect license plates in the video and determine their contours to access their size in the image. We went through two successive steps for this. First, we used the **YOLO model** again to detect license plates. In this case, we couldn't use the pre-trained version on the COCO dataset because license plates are not part of the classes in that dataset. Therefore, we opted for specific training using a **license plate detection dataset**. More specifically, this github repository [8] details every step on how to train the YOLO model on this dataset, using Google collab GPUs. Additionally, it provides the metrics of the training, such as the ROC curve and the confusion matrix. For this reason, we decided to not copy this work and use directly the trained model as found here [9]. This version of YOLO allows the detection of license plates in an image.

However, as the bounding boxes were not very accurate, using YOLO was not sufficient to access the size of the license plate. Therefore, we applied a series of image processing functions

from OpenCV to the frames to flatten and fill the bright area corresponding to the plate and detect its contours. We used the `morphologyEx` function with the `MORPH_CLOSE` parameter from OpenCV for this purpose. Note that we were particularly concerned with the accuracy of the **plate's width**, and the functions used were adjusted for this, even if it meant being slightly less precise in detecting the height (we assumed that, in practice, plates were seen horizontally and almost face-on to the camera).

To avoid false positives during this detection, we implemented several tests to limit the selected contours. Firstly, these tests required the **height-to-width ratio** of the detected contours to match that of a real license plate. Additionally, we verified that the areas of the detected zones were **not too small**.

Once the contours of the license plate are detected, we can calculate the distance separating it from the camera. Indeed, for a camera, we have the formula:

$$T = d \times T_{image} \times C$$

where T is the real size of the object, d is its distance to the camera, T_{image} is its size in the image (in pixels or as a percentage of the total size, for example), and C is a constant depending only on the camera parameters (focal distance, screen size, etc.).

It is possible to determine the factor C by taking a picture of an object of **known size** at a **known distance** from the camera (while fixing the focal length). Indeed, in that case, we have:

$$C = \frac{T_{ref}}{d_{ref} T_{imageref}}$$

and therefore :

$$d = d_{ref} \frac{T T_{imageref}}{T_{image} T_{ref}}$$

This is what we did with a phone and a ruler to get a plausible value for C . Even if it does not exactly correspond to the one we should use with videos from the internet, it gives coherent results.

For a European license plate, we have $T = 0.52 \text{ m}$. Assuming the plate is almost facing the camera, we can calculate d , and subsequently, the **distance to all vehicles** (along the view axis) by combining the previously mentioned algorithms.

3.5 POOLING AND CALIBRATION

We finally applied these methods to a **video**. We imported the video using the `VideoCapture` function from OpenCV, and for each frame, we applied the following algorithm:

- Detection of **the vanishing point** and recording of its coordinates on the image.
- Detection of **vehicles**.
- For each vehicle, detection of its **license plate**. If a plate is found, the distance to the vehicle is calculated, and reference data is stored.

- In all cases, application of the **distance calculation formula** on the road using reference data and its vanishing point.
- Computation of the **relative speed** of detected vehicles.
- Display on the frame of **bounding boxes** for vehicles associated with their distances and relative speeds, as well as **the vanishing point**.

Regarding the reference data (related to license plate detection), a question arose about the method to adopt before a plate is detected for calibration. We chose not to display distances and speeds in this case.

4 RESULTS

4.1 IMAGES

4.1.1 • DETECTION OF VEHICLES

The results given by the YOLO v8 model are very convincing. With relatively little effort, we are able to display a real-time video including **vehicle detections**, an **index for tracking**, and a **reliability rate**.

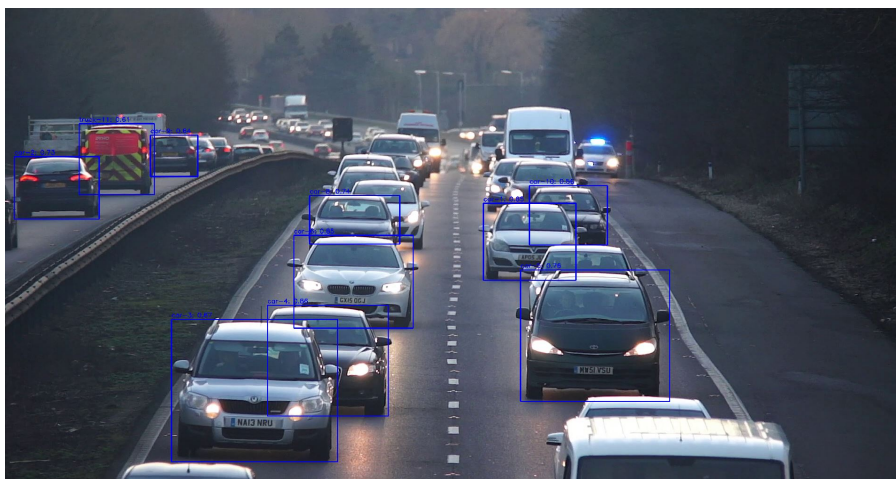


Figure 5: Car detection on a frame extracted from a video (high confidence threshold)

However, we noticed an issue that could distort distance calculations. In some cases, the bounding box of the vehicles includes their shadows, causing the bottom line to be **lower** than it should be.

4.1.2 • VANISHING POINT DETECTION

The vanishing point detection works in most cases. Here are some examples:

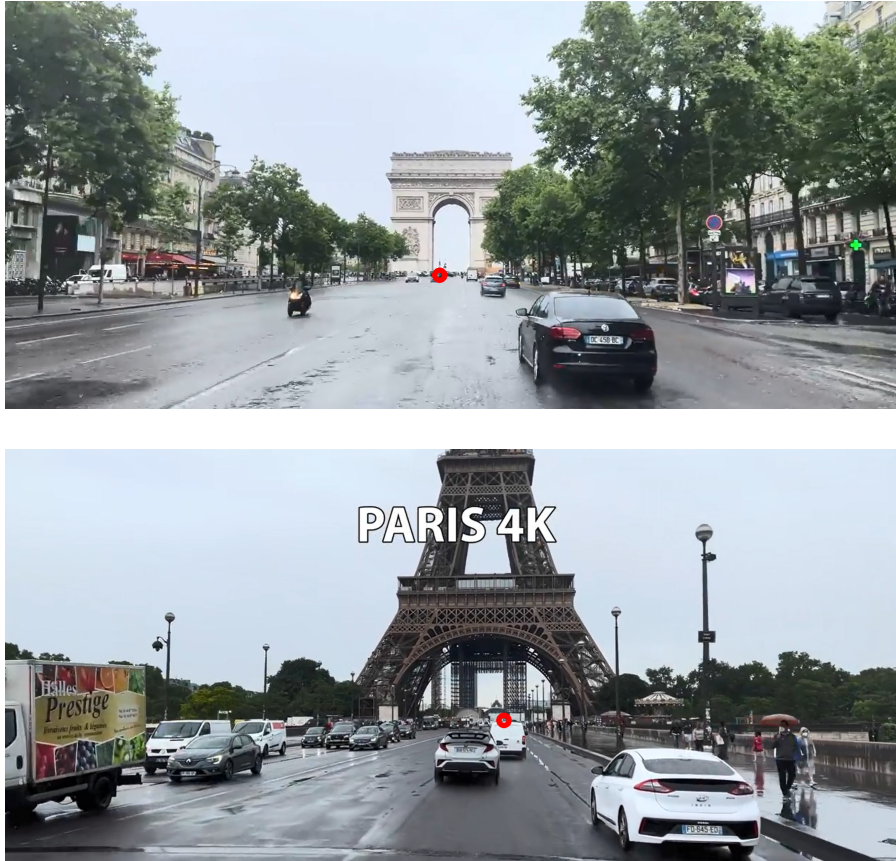


Figure 6: Example of successful detections (the red circle indicates the detected vanishing point)

Nevertheless, we have identified certain scenarios where it fails, especially on roads where **traffic lines are not visible** or in images with **very high vehicle density**. Unfortunately, these cases are likely to occur in real-world conditions and would require methods other than the one used for vanishing point detection, such as, perhaps, approaches involving machine learning.



Figure 7: Examples of failed detection (not enough lines found)

4.1.3 • LICENSE PLATES DETECTION

Just like the vanishing point detection, the license plate detection works in numerous cases:



Figure 8: Examples of successful detections (the plate is surrounded in green)

Nevertheless, depending on the **lighting conditions** of the plate and the car, as well as the **resolution** of the plate in the video, detection can fail or, worse, be inaccurate:



(a) No license plate detected (resolution too low)



(b) License plate detected inaccurately

Figure 9: Examples of failed detections (the plate is surrounded in green)

To check our calculations and algorithm, we conducted a test to estimate the distance based on the license plate (we took a picture with the phone we used to compute C):



Figure 10: Example of distance estimation

We find that the distance between the camera and the truck is approximately 8.93 meters, which is consistent with the actual distance of 9,5 meters.

4.2 VIDEOS

The method works with videos in the same way it works with images.

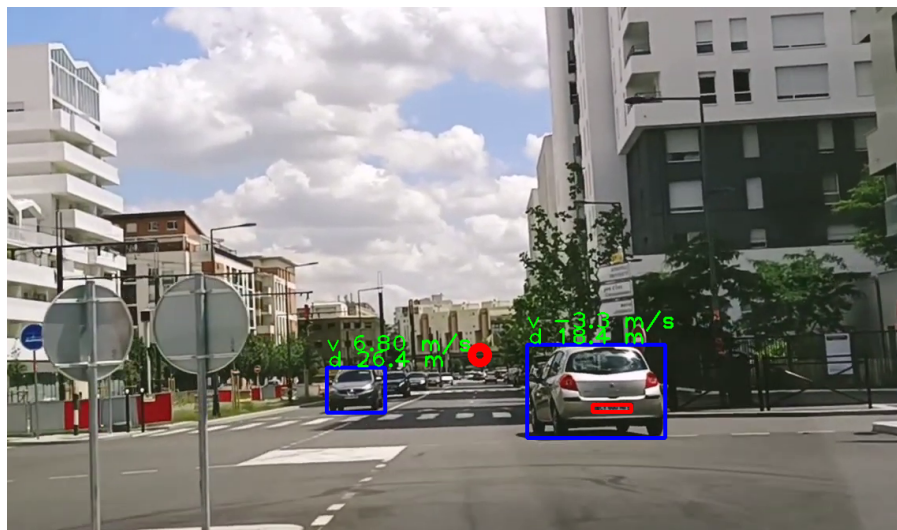


Figure 11: Example extracted from the video (vanishing point: circle, vehicle: blue box, license plate: red box)

However, as we have seen, if it generally performs well for clear roads, there are cases where license plate or vanishing point detection can be compromised due to specific conditions in the image: **poor lighting**, **blurry video**, the presence of **unwanted lines**, **erased traffic lines**, and more. In practice, the results obtained from a video are quite **noisy**.

To address this issue, we chose to calculate the **average** of results over several frames for three parameters: the position of the vanishing point of the road and the constant involved in distance calculations, and the speed. This helps achieve more stable results over time. For the first two parameters, we also decided to **discard results** that deviate too much from the values calculated in the previous frames to get rid of outliers.

The last issue arising with video applications is the computation time. Our algorithm is **too slow** to run in real-time (it processes approximately 5 frames per second). It's worth noting that the computation time increases with the number of detected vehicles. Several methods could be considered to improve this. For instance, we could perform calculations **only every 5 or 10 frames**, **reduce the video size**, or **parallelize** the computation for each vehicle.

5

CONCLUSION

Throughout this project, we have implemented an algorithm to **assess the distances and relative speeds** (along the view axis) between a car and other vehicles in front of it. This is achieved using **only a camera** placed in an arbitrary manner (yet approximately parallel to the road) to capture what a driver would see.

The algorithm consists of three main phases: **vehicle detection** in the video using YOLO, determination of **the vanishing point** using OpenCV's `HoughLinesP` method and a RANSAC algorithm, and **license plate contour detection** using YOLO and OpenCV's image processing functions. The results are satisfactory, and we manage to obtain a coherent estimation as the output of the algorithm.

However, the precision of this method remains **approximate**. Beyond the approximations that have been made, which could be corrected with a better knowledge of the camera used and pre-calibrations, the method is inherently uncertain. With just a few pixels difference at the time of license plate or vanishing point detection, the distances vary considerably, especially for longer ones. Consequently, speeds estimations are very noisy. Therefore, in a context where estimation errors could lead to life-threatening situations, it seems preferable to use **more advanced technologies**, such as LiDAR, for object detection around autonomous vehicles.

REFERENCES

- [1] Ahmed Ali et al (2020) *Real-time vehicle distance estimation using single view geometry*, KeepTruckin Inc. Lahore, Pakistan, https://openaccess.thecvf.com/content_WACV_2020/papers/Ali_Real-time_vehicle_distance_estimation_using_single_view_geometry_WACV_2020_paper.pdf
- [2] Xiaoyun An et al (2021) *Vanishing Point Detection based on Line Set Optimization*, J. Phys.: Conf. Ser. 1748 032052, <https://iopscience.iop.org/article/10.1088/1742-6596/1748/3/032052/pdf>
- [3] Kuoyuan Li (2021,December 27) *Detecting the vanishing point in one-point perspective images using Computer Vision algorithms*, Medium, <https://medium.com/@KuoyuanLi/detecting-the-vanishing-point-in-one-point-perspective-images-using-computer-vision-algorithms-c4352d4e6c3e>
- [4] Adrian Rosebrock (2020,September 21) *OpenCV: Automatic License/Number Plate Recognition (ANPR) with Python*, PyImageSearch, <https://pyimagesearch.com/2020/09/21/opencv-automatic-license-number-plate-recognition-anpr-with-python/>
- [5] Dr. Priyanto Hidayatullah (2022,November 12) *Vehicle Detection Tracking and Counting (YOLO)*, Youtube, <https://www.youtube.com/watch?v=JqgwGreySTA>
- [6] Rohith Gandhi (2018,July 9) *R-CNN, Fast R-CNN, Faster R-CNN, YOLO — Object Detection Algorithms*, TowardsDataScience, <https://towardsdatascience.com/r-cnn-fast-r-cnn-faster-r-cnn-yolo-object-detection-algorithms-36d53571365e>
- [7] Rohit Kundu (2023,January 17) *YOLO: Algorithm for Object Detection Explained [+Examples]*, v7labs, <https://www.v7labs.com/blog/yolo-object-detection>
- [8] jonG312 *YOLOv8 Vehicle Plate Recognition*, Github, <https://github.com/jonG312/YOLOv8-Vehicle-Plate-Recognition>
- [9] Muhammad-Zeerak-Khan *Automatic License Plate Recognition using YOLOv8*, Github, <https://github.com/Muhammad-Zeerak-Khan/Automatic-License-Plate-Recognition-using-YOLOv8>
- [10] Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi (2016,May 9) *You Only Look Once: Unified, Real-Time Object Detection*, Arxiv <https://doi.org/10.48550/arXiv.1506.02640>